

Graph Learning for Reliability and Dependability Analysis in Complex Distributed Systems

Abstract

Modern distributed software systems—including publish–subscribe middleware and microservice meshes—heavily decouple components in space and time. While this decoupling enables horizontal scalability and autonomous lifecycle management, it conceals the complex dependency pathways along which cascading failures propagate. Because operational telemetry does not exist prior to deployment, and source-code static analysis cannot observe distributed topology, identifying systemically critical components and explaining their vulnerability remains a fundamental challenge.

We present **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that models complex distributed architectures as typed, weighted, directed multigraphs over five entity classes (Applications, Brokers, Topics, Nodes, and Libraries) with logical dependencies derived through typed projection rules. On this representation, we formulate a relation-specific **Heterogeneous Graph Transformer (HGL)** to forecast cascading failure impact and detect critical components, complemented by an interpretable **Reliability–Maintainability (RM)** attribution baseline grounded in the ISO/IEC 25010/25019 quality models. Both predictors are evaluated against discrete-event cascade simulators operating on a structurally disjoint view of the system under a strict input–label independence guarantee.

Across seven synthetic topologies and three authentic open-source distributed systems (Autoware.universe ROS 2, a Cloud-Native Microservices mesh, and the Train-Ticket benchmark), our empirical evaluation yields four principal findings: **(1)** Typed graph learning consistently outperforms non-learning structural baselines, demonstrating its strongest advantage in critical-set identification ($F_1@K = 0.465$ vs. 0.308) and inductive out-of-distribution ranking ($\rho = 0.608$ vs. 0.521). **(2)** Relation-specific message passing proves crucial for cross-architectural generalisation, where typed GNNs outperform homogeneous graph attention networks by $+0.172$ in rank correlation under Leave-One-Scenario-Out validation. **(3)** The hierarchical RM profile successfully disentangles error propagation (fault tolerance) from single-point-of-failure vulnerabilities (availability), directing engineers toward targeted architectural remediations. **(4)** SaG successfully generalizes to complex real-world architectures, achieving strong rank agreement ($\rho = 0.688$ to 0.778) and robust critical-set containment across both cyber-physical and cloud-native paradigms.

Keywords: Graph representation learning, Heterogeneous graph neural networks, Publish–subscribe middleware, Distributed systems dependability, Cascading failures, Static system analysis, Architectural quality models

1. Introduction

1.1. Motivation

The publish–subscribe (pub-sub) and asynchronous event-driven paradigms have become core architectural abstractions for large-scale distributed systems, spanning cyber-physical systems, autonomous vehicles, cloud-native microservices, robotics, and the Internet of Things (IoT). By decoupling producers and consumers in time, space, and synchronization, pub-sub architectures allow independent components to scale, evolve, and fail without direct coordination [1]. Standardized middleware—such as the Data Distribution Service (DDS) [2], MQTT [3], Apache Kafka [4], and ROS 2 [5]—exposes rich deployment-time Quality-of-Service (QoS) configurations (including reliability, durability, and deadline policies) that govern message routing and system behavior under stress.

However, the very decoupling that makes asynchronous distributed systems flexible also creates a profound visibility barrier when reasoning about dependability. In pub-sub and event-driven architectures, there are no explicit caller–callee invocations: a publishing process maintains no direct static reference to its consumers, even when those consumers are entirely dependent upon its data stream. Consequently, cascading failures do not follow traditional call graphs. Instead, failures propagate along *derived* logical paths across message brokers, shared topics, colocated execution nodes, and shared software libraries. When a shared library crashes, it induces a *simultaneous failure* across all consuming applications rather than a sequential cascade. Traditional architectural block diagrams and static call graphs fail to reveal these multi-layer failure pathways, meaning that the components whose failure would cause the most catastrophic downstream outage are rarely the ones that appear prominent in high-level diagrams.

Addressing this vulnerability is most cost-effective *prior to deployment*, during the design and continuous integration (CI/CD) stages, where structural refactoring and redundancy insertion are cheapest. Yet, pre-deployment is precisely the phase where no runtime telemetry, distributed tracing, or operational logs exist. Software architects and reliability engineers must therefore answer two critical questions solely from design-time descriptors:

1. *Which components and communication links are systemically critical to system availability and reliability?*
2. *Why are they critical, and what specific architectural intervention (e.g., broker replication, topic decoupling, library sandboxing) will most effectively mitigate that risk?*

Uncontained cascading failures in production also carry substantial sustainability and operational costs: emergency failovers, retransmission storms, and compute-heavy restart loops waste energy and infrastructure resources that proactive architectural hardening avoids.

1.2. Problem Statement and Limitations of Existing Work

We formulate pre-deployment dependability analysis for complex distributed systems as two coupled tasks:

1. **Interpretable Criticality Attribution:** Computing a multi-dimensional, standards-grounded quality profile for every component and dependency—derived from ISO/IEC 25010 [6] and ISO/IEC 25019 [7]—to explain *how* and *why* a component is vulnerable.
2. **Failure-Impact Forecasting:** Accurately predicting the global cascade blast radius and ranking the most failure-critical components using learned topological representations, enabling engineers to prioritize hardening under budget constraints.

Existing techniques fall short of bridging what we term the “**Architecture–Code Gap**”: a distributed system may exhibit pristine, bug-free source code within each component, yet remain systemically fragile due to hidden topological single points of failure (SPOFs) or mismatched middleware QoS contracts. Three prevailing paradigms leave this gap unaddressed:

- **Static Code Analysis (SCA):** Tools such as SonarQube evaluate code complexity [8], cohesion, and modularity [9, 10] at the individual service level. However, SCA is entirely blind to inter-component distributed topology, middleware message queues, and cross-host failure propagation.
- **Runtime Chaos Engineering and Telemetry:** Approaches such as Chaos Monkey [11] and distributed tracing inject real faults into running staging or production clusters. While valuable, they require fully operational infrastructure, carry operational risks, and operate too late in the software development lifecycle to guide design-time architecture.
- **Homogeneous and Topology-Only Network Analysis:** Traditional graph centrality measures (betweenness, PageRank, degree) [12, 13, 14, 15] collapse complex systems into flat, unweighted networks. They discard node and edge semantics, conflating entirely distinct failure modes (such as sequential event propagation vs. simultaneous library crashes). Similarly, homogeneous Graph Neural Networks (GNNs) [16, 17, 18] flatten heterogeneous middleware interactions, losing relation-specific propagation patterns.

No existing approach provides a unified, pre-deployment framework that combines typed multigraph representations, code-level quality ingestion, heterogeneous graph learning, and interpretable quality attribution.

1.3. The Software-as-a-Graph (SaG) Approach

To overcome these limitations, we introduce **Software-as-a-Graph (SaG)**, a pre-deployment **Static System Analysis (SSA)** framework. SaG operates on system architecture descriptions (expressed as Architecture-as-Code manifests) and executes a four-stage analysis pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed system as a typed, weighted, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic **DEPENDS_ON** dependency layer that explicitly captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Interpretable Quality Attribution:** SaG integrates code-level SCA metrics with topological properties into a hierarchical **Reliability–Maintainability (RM)** attribution model (Section 4). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (structural articulation and SPOF exposure), providing explainable diagnostics for reliability engineers.
4. **Graph Learning for Failure Forecasting:** SaG deploys a **Heterogeneous Graph Transformer (HGL)** that performs relation-specific attention and message passing across typed entities to forecast cascading failure impacts (Section 5).

To guarantee methodological rigor, the framework enforces an **input–label independence guarantee**: the learned and attribution predictors operate strictly on the derived analytical graph G_{analysis} , whereas ground-truth failure impacts are generated by discrete-event simulators executing over the raw structural topology $G_{\text{structural}}$ (Section 5.3).

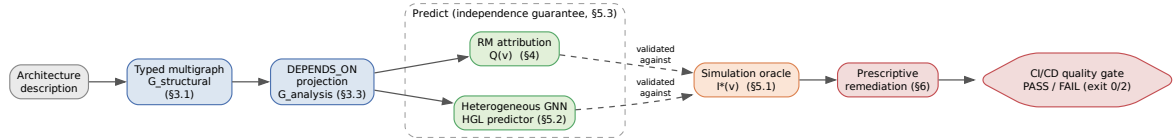


Figure 1: End-to-end architecture of the Software-as-a-Graph (SaG) framework: architecture manifest ingestion → typed multigraph → logical dependency projection → dual predictor paths (interpretable RM baseline and Heterogeneous Graph Transformer) validated against independent discrete-event simulation.

1.4. Research Questions

Our investigation evaluates the predictive power, interpretability, and generalizability of SaG across four research questions:

RQ1 (Predictive Efficacy): How accurately does typed graph learning predict cascading failure impact and identify critical component sets compared to non-learning topological baselines, both in-distribution and under inductive cross-scenario evaluation?

RQ2 (Value of Heterogeneity): What failure mechanisms and structural vulnerabilities does typed heterogeneity reveal that homogeneous graph representations and single-scalar centralities obscure?

RQ3 (Ablations and Sensitivity): How do explicit QoS feature encodings and multi-attribute weighting calibrations affect model performance and ranking stability?

RQ4 (Real-World Generalization): Does the graph learning framework successfully transfer to authentic, real-world open-source distributed software architectures across different architectural paradigms?

1.5. Key Contributions

This paper makes the following primary contributions:

1. **A Formal Typed Architecture Model:** A multigraph representation of distributed pub-sub and event-driven systems that derives logical dependencies and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures (Section 3).
2. **Heterogeneous Graph Learning for Dependability:** A relation-specific Heterogeneous Graph Transformer (HGL) tailored for pre-deployment cascading failure prediction and critical component detection (Section 5).
3. **Standards-Grounded Interpretable Attribution:** An explainable Reliability–Maintainability (RM) quality baseline grounded in ISO/IEC 25010/25019 that bridges code-level SCA metrics with system-level topological criticality (Section 4).
4. **Empirical Evaluation and Scope Conditions:** A rigorous empirical evaluation across seven synthetic topologies (1,545 components) and three real-world open-source systems (Autoware.universe ROS 2, Cloud-Native Microservices, and Train-Ticket; 225 components) under strict input–label independence, establishing where typed graph learning provides decisive advantages (Sections 6–7).

1.6. Paper Organization

The remainder of this paper is structured as follows: Section 2 reviews related work in distributed systems dependability, static system analysis, and graph representation learning. Section 3 formalizes the Software-as-a-Graph architectural model and dependency projection rules. Section 4 presents the interpretable RM attribution baseline. Section 5 details the Heterogeneous Graph Transformer formulation and simulation ground truth. Section 6 describes the experimental setup, datasets, and protocols. Section 7 presents empirical results for RQ1–RQ4. Section 8 discusses architectural implications, threats to validity, and conclusions.

2. Related Work

This research intersects four major domains: distributed systems dependability, static system analysis, graph representation learning for network vulnerability, and software quality models.

2.1. Dependability in Distributed and Pub-Sub Systems

The publish–subscribe paradigm provides foundational communication decoupling for distributed systems [1]. Formal middleware specifications—such as DDS [2], MQTT [3], ROS 2 [5], and distributed commit logs like Kafka [4]—enable fine-grained QoS policies governing durability, liveness, and transport reliability. Prior research on pub-sub dependability has predominantly focused on runtime mechanisms: fault-tolerant event routing, broker clustering, dynamic consensus, and adaptive message retransmission.

In parallel, runtime verification and chaos engineering [11] systematically inject faults into staging or production clusters to observe steady-state recovery. While essential for validating operational resilience, runtime techniques operate late in the software lifecycle, require complete infrastructure deployments, and cannot evaluate architectural alternatives during initial design. Our work addresses the complementary, pre-deployment phase: predicting cascading vulnerability from architectural models *before* systems are deployed.

2.2. Static Code Analysis vs. Static System Analysis

Traditional Static Code Analysis (SCA) analyzes source code abstract syntax trees (ASTs) to measure cyclomatic complexity [8], module coupling (e.g., LCOM, CBO) [9, 10], and code duplication. While SCA effectively identifies internal code smells and defect-prone modules [19, 20, 21, 22], it is entirely oblivious to inter-process communication, container placement, and middleware topologies.

To bridge this “Architecture–Code Gap,” **Static System Analysis (SSA)** elevates static analysis to the system architecture level. By modeling distributed components, message channels, and infrastructure hosts as a global dependency graph, SSA frameworks propagate code-level quality attributes across topological links. This enables continuous pre-deployment verification in modern CI/CD pipelines [23, 24], catching structural anti-patterns [25, 26, 27, 28] and architectural degradation [29, 30] at commit time.

2.3. Network Science and Graph Representation Learning

Network science offers established metrics for identifying critical network elements, including degree, closeness, betweenness centrality [12, 14], articulation points, and PageRank [13, 15]. Studies on random failure tolerance [31], cascading overload [32], and interdependent networks [33] have provided deep mathematical foundations for systemic vulnerability. However, standard network metrics suffer from two fundamental limitations when applied to software architectures:

1. *Dimensional Collapse*: A single centrality score cannot distinguish a structural Single Point of Failure (SPOF) from an error-propagating cascade hub or a high-maintenance bottleneck.
2. *Semantic Collapse*: Discarding node and edge types treats an asynchronous pub-sub topic the same as a physical host or a shared library, masking distinct failure propagation mechanics.

To overcome the limitations of hand-engineered graph metrics, recent research has applied machine learning on graphs for network dismantling and criticality estimation (e.g., FINDER [34], DrBC [35], and PowerGraph [36]). However, most existing models rely on homogeneous message passing (GCN [16], GraphSAGE [17], GAT [18]). Because distributed middleware is inherently multi-typed and heterogeneous, homogeneous aggregation mixes fundamentally incompatible semantic relationships. Heterogeneous Graph Neural Networks (RGCN [37], HAN [38], HGT [39], MAGNN [40]) address this by parameterizing relation-specific transformations. We build upon the Heterogeneous Graph Transformer (HGT) architecture [39] to maintain typed relational semantics during cascade impact forecasting.

2.4. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the ISO/IEC 25010:2023 product quality model [6] (comprising nine characteristics including Reliability, Maintainability, and Security) and the ISO/IEC 25019:2023 Quality-in-Use standard [7] (evaluating stakeholder harm over Beneficialness, Freedom from Risk, and Acceptability). Quality evaluation distinguishes between *internal quality* (measured statically on software artifacts at rest) and *external quality* (measured dynamically on executing systems) [41, 42].

Synthesizing multi-attribute structural metrics into actionable quality indices constitutes a classic Multi-Criteria Decision Making (MCDM) problem. The Analytic Hierarchy Process (AHP) [43] provides a mathematically rigorous pairwise-comparison framework equipped with an explicit Consistency Ratio ($CR \leq 0.10$) to validate expert weighting schemes. In this work, we use AHP to construct an audited, interpretable Reliability–Maintainability attribution baseline that serves as a transparent benchmark for our learned GNN models.

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), and the dual graph views utilized throughout the framework (Section 3.3).

3.1. Formal Multigraph Definition

We model a complex distributed system as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- V is the set of system entities, partitioned into five disjoint types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \quad (2)$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \rightarrow [0, 1]$ and $w_E : E \rightarrow [0, 1]$ are weighting functions representing entity criticality and coupling strength.

Table 1: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Node (V_{node})	Physical host or virtualized environment	Bare-metal server, Kubernetes worker, Cloud V
Library (V_{lib})	Shared software package or driver dependency	<code>librdkafka</code> , OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO	App/Library $\rightarrow$ Topic	Component publishes messages to topic
SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component consumes messages from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Node	Process is hosted on physical/virtual host
CONNECTS_TO	Node $\rightarrow$ Node	Physical network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library dependency

Application and Library entities additionally ingest static code metrics computed via SCA tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), directly bridging code-level fragility with topological analysis.

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links exhibit varying degrees of coupling based on their Quality-of-Service (QoS) contracts. For example, a **RELIABLE** topic with **TRANSIENT_LOCAL** durability binds communicating services far more tightly than a **BEST_EFFORT** telemetry stream.

For each structural communication edge e , we compute an edge coupling weight $w_E(e) \in [0, 1]$ derived from the QoS profile of the mediating topic:

$$w_E(e) = 0.85 \cdot (w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}) + 0.15 \cdot q_{\text{payload}} \quad (3)$$

where $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport priority scores, with AHP weights (0.30, 0.40, 0.30) ($CR < 0.05$). Durability is assigned the highest weight because it dictates message persistence across network partitions. A minimum floor $w_E(e) \geq 0.01$ ensures that best-effort edges remain visible to graph traversals.

Table 2: The six `DEPENDS_ON` logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	<code>app_to_app</code>	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive <code>USES</code>)	$\max_t w_E(t)$
2	<code>app_to_broker</code>	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$\max_t w_E(t)$
3	<code>node_to_node</code>	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	Lifted $\max w$
4	<code>node_to_broker</code>	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	Lifted $\max w$
5	<code>app_to_lib</code>	Application $\rightarrow$ Shared Library it <code>USES</code>	$w_V(\text{app})$
6	<code>broker_to_broker</code>	Broker $\leftrightarrow$ Broker (bidirectional, colocated on same host)	$w_V(\text{node})$

3.2.1. Logical Dependency Projection (`DEPENDS_ON`)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We derive a single unified semantic relation, `DEPENDS_ON`, directed from *dependent* to *dependency* (“if target fails, source is impacted”):

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A key insight of the SaG model is distinguishing between two fundamentally different failure modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers experience data starvation. The failure propagates step-by-step through topics and message queues.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single event.

Retaining entity types and relation-specific dependency rules allows SaG to model both mechanisms, whereas untyped graphs collapse them into identical edges.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw graph containing physical and structural edges (`PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, `USES`). This view is consumed exclusively by discrete-event simulators to execute unbiased failure injections (Section 5.1).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis} .

G_{analysis} is further organized into four analytical layers (Application, Middleware, Infrastructure, and Global System), allowing architects to evaluate criticality at subsystem levels (e.g., following MIL-STD-498 hierarchical structures).

4. Interpretable Attribution as a Quality Baseline

To provide actionable architectural explanations alongside data-driven predictive rankings, SaG decomposes component and relationship criticality into a multi-dimensional, standards-grounded quality attribution profile.

4.1. Grounding in ISO/IEC Standards

Following **ISO/IEC 25010:2023** (Product Quality Model) [6] and **ISO/IEC 25019:2023** (Quality-in-Use) [7], we formulate two core formal criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system’s capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated primarily across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**.

Table 3: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics
Reliability (R)	Fault Tolerance (FT)	How broadly does failure propagate?	Reverse PageRank on $G^\top$, in-deg
	Availability (A)	Is this a single point of failure?	Directed articulation score, bridge
Maintainability (M)	Modularity/Modifiability	How complex and coupled is this?	Betweenness, out-degree, Code F

Coverage Scope: SaG focuses on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, e.g., ISO 26262 ASIL ratings) and Security (which requires explicit threat models, e.g., STRIDE) are declared external to purely structural analysis and are left for domain-specific extensions.

4.2. Typed Node Feature Representation

SaG extracts rich feature vectors tailored to the 5 distinct entity types in the software multigraph:

- **Application ($|V_{\text{app}}|$, 23 dims):** Indices 0–17 represent shared topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load). Indices 18–22 capture source code metrics extracted via Static Code Analysis (SCA): Lines of Code (LOC), Cyclomatic Complexity, Martin’s Instability metric ($I_{\text{code}} = \frac{C_e}{C_a + C_e}$), Lack of Cohesion in Methods (LCOM), and composite Code Quality Penalty (CQP).
- **Library ($|V_{\text{lib}}|$, 23 dims):** Same shared topological (0–17) and code quality (18–22) metrics.
- **Broker ($|V_{\text{broker}}|$, 19 dims):** Indices 0–17 shared topological metrics; index 18 represents normalized queue buffer capacity.
- **Topic ($|V_{\text{topic}}|$, 22 dims):** Indices 0–17 shared topological metrics; indices 18–21 capture publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node ($|V_{\text{node}}|$, 20 dims):** Indices 0–17 shared topological metrics; indices 18–19 capture normalized CPU core allocation and physical memory (RAM).

4.3. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. The quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [43]:

1. **Fault Tolerance ($FT(v)$):** Measures error cascade potential on the transpose graph $G_{\text{analysis}}^\top$ (where edges follow failure propagation from dependency to dependent):

$$FT(v) = w_{\text{rpr}} \cdot \text{RPR}(v) + w_{\text{indeg}} \cdot \text{Deg}_{\text{in}}(v) + w_{\text{depth}} \cdot \text{DepthPotential}(v) \quad (4)$$

where $w_{\text{rpr}} = 0.50$, $w_{\text{indeg}} = 0.30$, and $w_{\text{depth}} = 0.20$.

261 2. **Availability** ($A(v)$): Identifies structural single points of failure (SPOFs):

$$A(v) = w_{\text{art}} \cdot \text{ArticulationScore}(v) + w_{\text{bridge}} \cdot \text{BridgeCentrality}(v) \quad (5)$$

262 where $w_{\text{art}} = 0.65$ and $w_{\text{bridge}} = 0.35$.

263 3. **Reliability** ($R(v)$): Blends Fault Tolerance and Availability hierarchically:

$$R(v) = \alpha \cdot FT(v) + (1 - \alpha) \cdot A(v), \quad \alpha = 0.36 \quad (6)$$

264 Intra-dimension pairwise comparison matrices are audited to guarantee AHP consistency ($CR \leq 0.08$).

265 4. **Maintainability** ($M(v)$): Evaluates structural coupling combined with code-level static analysis:

$$M(v) = 0.50 \cdot \text{Betweenness}(v) + 0.35 \cdot \text{Deg}_{\text{out}}(v) + 0.15 \cdot \text{CodeQualityPenalty}(v) \quad (7)$$

266 The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \quad (8)$$

267 When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is
268 reweighted dynamically:

$$Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v) \quad (9)$$

269 Components are categorized into adaptive criticality tiers using box-plot quartile thresholds:

- 270 • **CRITICAL:** $Q > Q_3 + 1.5 \cdot \text{IQR}$
- 271 • **HIGH:** $Q_3 < Q \leq Q_3 + 1.5 \cdot \text{IQR}$
- 272 • **MEDIUM:** $Q_1 < Q \leq Q_3$
- 273 • **MINIMAL:** $Q \leq Q_1$

274 This enables actionable diagnostics: a service scoring high on A but low on FT is diagnosed as a pure
275 SPOF requiring horizontal replication, whereas a service scoring high on FT is an error cascade hub requiring
276 circuit breakers and bulkhead isolation.

277 5. Graph Learning for Failure-Impact Prediction

278 While the interpretable RM baseline provides explainable quality profiles, complex non-linear failure
279 interactions and non-local cascade propagations benefit significantly from graph representation learning. This
280 section details our Heterogeneous Graph Transformer (HGT) architecture, 16-dimensional edge representation,
281 multi-task prediction heads, dimension-masked loss formulation, ground-truth simulation oracles, and the
282 strict input-label independence guarantee.

283 5.1. Ground-Truth Simulation Oracles

284 To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry,
285 SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a
286 formal three-oracle taxonomy:

- 287 • **Cascade Reachability Oracle** ($I^*(v)$): Implemented via **FaultInjector**, this oracle injects an
288 unexpected node crash at component $v \in V$, propagates cascading failures across dependent topics,
289 brokers, and network links using breadth-first dynamic traversal, and computes the fraction of surviving
290 subscriber feeds severed by the outage. $I^*(v) \in [0, 1]$ serves as the primary continuous target label for
291 training and evaluating GNN predictors.

- **Multi-Metric Composite Oracle ($I_{\text{comp}}(v)$):** Implemented via `FailureSimulator`, this oracle evaluates a multi-faceted failure impact vector:

$$I_{\text{comp}}(v) = 0.35 \cdot \Delta\text{Reachability} + 0.25 \cdot \Delta\text{Fragmentation} + 0.25 \cdot \Delta\text{Throughput} + 0.15 \cdot \Delta\text{FlowDisruption} \quad (10)$$

$I_{\text{comp}}(v)$ serves as the canonical oracle for architectural quality gate verification.

- **Dynamic Queue-Flow Oracle ($I_{\text{dyn}}(v)$):** Implemented via `MessageFlowSimulator` using the SimPy discrete-event simulation framework [44], this oracle models message emission rates, stochastic network latencies, broker buffer saturation, and dropped delivery counts under fault injection.
- **Relationship (Edge) Removal Oracle ($I_{\text{edge}}(u, v)$):** Evaluates the systemic impact of severing an individual dependency or communication channel while keeping both endpoint components alive:

$$I_{\text{edge}}(u, v) = I_{\text{comp}}(G \setminus \{(u, v)\}) - I_{\text{comp}}(G) \quad (11)$$

5.2. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (`CALLS`, `PUBLISHES_TO`, `SUBSCRIBES_TO`, `HOSTED_ON`, etc.), we implement a 3-layer **Heterogeneous Graph Transformer (HGT)** architecture [39] with hidden dimension $D = 64$ and 4 attention heads.

5.2.1. Edge Feature Encoding (16-Dimensional)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$:

- **Index 0:** Scalar QoS weight $w(e) \in (0, 1]$, computed via the harmonic mean of reliability, durability, priority, deadline, and blocking multipliers.
- **Index 1:** Normalized path count through edge e in G_{analysis} .
- **Indices 2–8:** 7-bit one-hot encoding for edge relationship types (`CALLS`, `PUBLISHES_TO`, `SUBSCRIBES_TO`, `HOSTED_ON`, `RUNS_ON`, `DEPENDS_ON_LIB`, `CONNECTS_TO`).
- **Indices 9–15:** 7 explicit QoS profile dimensions: (9) Reliability policy (0 = best-effort, 1 = reliable); (10) Durability policy (0 = volatile, 1 = transient local, 2 = transient, 3 = persistent); (11) Normalized message priority; (12) Deadline constraint active flag (0/1); (13) Log deadline nanoseconds $\log_{10}(1 + \text{deadline_ns})$; (14) Log max blocking time $\log_{10}(1 + \text{max_blocking_ms})$; and (15) QoS impedance mismatch flag (1 if upstream/downstream QoS contracts clash, 0 otherwise).

The `EdgeFeatureEncoder` projects e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}}e_{uv}$. Prior to relational attention, the edge projection is added directly to the destination node representation: $\tilde{h}_v = h_v + e'_{uv}$.

5.2.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v with meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (dimensions 19–23 depending on $\tau(v)$) are mapped into D -dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v)) \quad (12)$$

2. **Relational Mutual Attention:** Query, Key, and Value projections compute relation-specific attention across heads:

$$\text{Attn}(u, e, v) = \text{Softmax}_{\forall u \in \mathcal{N}(v)} \left(\frac{K(u)W_{\text{att}, \phi(e)}Q(\tilde{v})^\top}{\sqrt{D/H}} \right) \quad (13)$$

$$\text{Msg}(u, e, v) = V(u)W_{\text{msg}, \phi(e)} \quad (14)$$

- 327 **3. Bidirectional Message Passing:** To capture downstream backpressure and upstream cascading
 328 failures, message passing is executed over both forward and reverse relations (G_{analysis} and $G_{\text{analysis}}^\top$).
 329 **4. Residual Aggregation and Layer Normalization:**

$$h_v^{(l)} = \text{LayerNorm} \left(h_v^{(l-1)} + \text{Dropout} \left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v) \right) \right) \quad (15)$$

5.3. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG branches into specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

5.3.1. Dimension-Masked Loss Formulation

The joint optimization objective balances regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.1 \cdot \mathcal{L}_{\text{consistency}} + 0.3 \cdot \mathcal{L}_{\text{edge}} \quad (16)$$

where $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is ListMLE ranking loss [45], $\mathcal{L}_{\text{pairwise}}$ is margin-ranking loss, and $\mathcal{L}_{\text{consistency}} = \text{MSE}(\hat{I}^*(v), 0.8\hat{R}(v) + 0.2\hat{M}(v))$.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via **FaultInjector**) observes runtime failure reachability rather than static code maintainability, maintainability ground truth is unobserved during dynamic simulation. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R, M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \quad (17)$$

This prevents the unmeasured maintainability head from being penalized or driven toward zero by backpropagation.

5.3.2. Domain-Rewighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^\top$), the composite score is evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \quad (18)$$

where $M_{\text{static}}(v)$ is drawn directly from the structural analyzer’s maintainability baseline (y_{rm} maintainability column), combining learned dynamic reliability with static source-code maintainability.

5.4. Input-Label Independence Guarantee

To guarantee experimental rigor and eliminate data leakage, SaG enforces strict architectural decoupling:

- **Feature Space:** Derived exclusively from G_{analysis} (using static structural topology, SCA metrics, and declared QoS contracts).
- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (**FaultInjector**, **FailureSimulator**, **MessageFlowSimulator**).

No simulation outputs are ever exposed as input features to the GNN or attribution baseline.

Table 4: Experimental evaluation corpus.

Dataset / Architecture	System Paradigm	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	80	16	28	4	8	24	196
Enterprise Pub-Sub	Kafka Event Mesh	300	60	110	10	30	90	784
Financial Trading	Low-Latency Pub-Sub	60	12	22	4	6	16	148
Healthcare Integration	HL7/FHIR Event Mesh	50	10	18	4	6	12	122
Hub-and-Spoke Enterprise	Broker-Centric Messaging	70	14	24	6	8	18	172
IoT Smart City	MQTT Telemetry Mesh	200	40	75	8	20	57	512
Microservices Mesh	Cloud-Native Services	90	18	32	6	10	24	226
Autoware.universe [46]	Real-World ROS 2 Autoware	75	32	24	3	6	10	184
Cloud Microservices [47]	Real-World GCP Boutique	60	22	20	4	6	8	142
Train-Ticket [48]	Real-World Microservices	90	41	30	3	8	8	218

6. Experimental Setup

6.1. Datasets and System Corpus

Our evaluation corpus comprises 1,770 components across ten distributed system architectures:

The seven synthetic scenarios are generated using statistical topology generators spanning diverse degree distributions, clustering coefficients, and middleware QoS configurations. The three real-world architectures are hand-transcribed from open-source repositories using dedicated architectural adapters.

6.2. Baselines and Evaluated Predictors

We compare five primary predictor configurations:

1. **Topo-BL**: Unweighted structural centrality (betweenness centrality and articulation point scoring).
2. **Topo-QoS**: QoS-weighted topological centrality baseline.
3. **RM / $Q(v)$** : Deterministic hierarchical quality attribution baseline (Section 4).
4. **GL / GL-QoS**: Homogeneous Graph Attention Networks (GAT) [18] trained on the flattened, untyped graph projection.
5. **HGL / HGL-QoS**: Relation-specific Heterogeneous Graph Transformers (Section 5).

6.3. Evaluation Metrics and Protocols

- **Ranking Accuracy**: Spearman rank correlation (ρ) and Kendall’s tau (τ) between predicted rankings and ground-truth simulated impact $I^*(v)$.
- **Critical-Set Identification**: $F_1@K$, Precision@ K , and Recall@ K for top- K critical components, where $K = \lceil 0.20 \cdot |V_{\text{app}}| \rceil$.
- **Statistical Significance**: Paired Wilcoxon signed-rank tests [49] ($p < 0.05$) and bootstrap 95% confidence intervals ($B = 2,000$) [50, 51].

6.3.1. Evaluation Protocols

- **In-Distribution Evaluation**: 60% train / 20% validation / 20% test node splits pinned by node identity within each scenario, evaluated over five random seeds {42, 123, 456, 789, 2024}.
- **Inductive Leave-One-Scenario-Out (LOSO)**: Models are trained on six scenarios and evaluated zero-shot on the seventh unseen scenario to test out-of-distribution generalizability.
- **Real-World Architectural Transfer**: Evaluating zero-shot transfer on authentic open-source architectures.

7. Results and Empirical Analysis

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 5 presents the in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across all seven synthetic scenarios.

Table 5: In-distribution held-out Spearman rank correlation (ρ) against $I^*(v)$ (seed means over 5 seeds with bootstrap 95% CIs; n is held-out Application count).

Scenario	n	Topo-BL	Topo-QoS	GL	GL-QoS	HGL	HGL-QoS
AV System	16	0.308	0.750	0.760	0.655	0.713	0.692
Enterprise	60	0.393	0.797	0.853	0.513	0.885	0.883
Financial Trading	12	0.246	0.709	0.851	0.874	0.882	0.903
Healthcare	10	-0.182	0.772	0.815	0.804	0.842	0.845
Hub-and-Spoke	14	0.299	0.511	0.494	0.475	0.537	0.557
IoT Smart City	40	-0.063	0.068	0.674	0.474	0.891	0.883
Microservices	18	0.302	0.556	0.524	0.436	0.362	0.354
Mean	—	0.186	0.595	0.710	0.604	0.730	0.731

Table 6: Paired Wilcoxon signed-rank tests across scenarios ($n = 7$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p -value	Significance
HGL vs. Topo-BL	+0.544	7/7	0.0	0.016	Statistically Significant ($p < 0.05$)
HGL vs. GL-QoS	+0.126	6/7	5.0	0.156	Not significant
HGL vs. Topo-QoS	+0.135	5/7	8.0	0.375	Not significant
GL vs. Topo-QoS	+0.116	5/7	5.0	0.156	Not significant
HGL vs. GL	+0.020	5/7	11.0	0.688	Not significant
HGL-QoS vs. HGL	+0.001	3/7	13.0	0.938	Not significant

7.1.1. Out-of-Distribution (LOSO) Generalization

Under inductive Leave-One-Scenario-Out (LOSO) cross-validation, the model must predict cascading criticality on an unseen system topology:

Table 7: Inductive Leave-One-Scenario-Out (LOSO) evaluation.

Model Variant	Mean LOSO ρ	Std ρ	Critical-Set $F_1@K$	Requires Training
Topo-BL	0.105	0.151	0.179	No
Topo-QoS	0.521	0.305	0.308	No
RM / $Q(v)$ Baseline	0.181	0.198	0.264	No
GL (Homogeneous)	0.436	0.120	0.440	Yes
GL-QoS (Homogeneous)	0.430	0.125	0.435	Yes
HGL (Typed Heterogeneous)	0.608	0.177	0.465	Yes
HGL-QoS (Typed + QoS)	0.595	0.190	0.461	Yes

Key Insights for RQ1:

- Critical-Set Detection Advantage:** HGL achieves a 51% relative improvement in critical-set identification ($F_1@K = 0.465$ vs. 0.308 for Topo-QoS).
- Out-of-Distribution Superiority:** Under LOSO validation, HGL substantially outperforms non-learning baselines ($\rho = 0.608$ vs. 0.521) and homogeneous GNNs ($\rho = 0.436$).

3. In-Distribution Scope Condition: In-distribution, both HGL ($\rho = 0.730$) and GL ($\rho = 0.710$) outperform structural baselines on point estimates, but their margin over Topo-QoS is constrained by topology variance.

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the contribution of node and edge typing, we compare relation-specific HGL against homogeneous GL across different validation regimes:

- **In-Distribution:** Heterogeneous HGL leads homogeneous GL by $\Delta\rho = +0.020$ (0.730 vs. 0.710). On familiar topologies, homogeneous GNNs can approximate type distinctions through structural degree signatures.
- **Out-of-Distribution (LOSO):** Heterogeneous HGL outperforms homogeneous GL by $+0.172$ ($\rho = 0.608$ vs. 0.436) and exhibits substantially higher stability across folds. When encountering unseen topologies, relation-specific message passing is essential for generalizing failure mechanics.

7.2.1. Empirical Edge-Removal Analysis

We further probed edge criticality by simulating the removal of candidate relationship channels while keeping both endpoint components alive. On the `av_system` topology across 50 candidate bridge edges, 46 edges exhibited zero downstream cascade impact, while the 4 edges with non-zero impact were direct library communication channels (`PUBLISHES_TO` / `SUBSCRIBES_TO`). This demonstrates that individual communication links are largely redundant, whereas component-level failures and shared library dependencies induce disproportionate systemic disruption.

7.3. RQ3: Ablations and Sensitivity Analysis

7.3.1. QoS Feature Encoding

Explicitly encoding continuous QoS attributes in the GNN (*HGL-QoS*) yielded negligible differences compared to the base typed model (in-distribution: 0.731 vs. 0.730; LOSO: 0.595 vs. 0.608). The derived `DEPENDS_ON` topology already embeds the primary routing and coupling effects, rendering explicit QoS edge features largely redundant when structural typing is available.

7.3.2. Intra-Dimension AHP Weight Shrinkage

We evaluated the sensitivity of the RM attribution baseline by sweeping a shrinkage parameter $\lambda \in [0, 1]$ blending internal term weights from uniform prior ($\lambda = 0$) to calibrated AHP judgement ($\lambda = 1$):

Table 8: Sensitivity of RM rank correlation to AHP shrinkage parameter λ .

λ Setting	0.00 (Uniform)	0.50	0.70 (Default)	0.80	1.00 (Raw AHP)
Mean Rank Correlation (ρ)	-0.0512	-0.0254	-0.0188	-0.0140	-0.0067

As illustrated in Figure 4, the calibrated AHP judgement ($\lambda = 1.00$) outperforms uniform weighting monotonically by $+0.045\rho$, validating the internal consistency of the expert hierarchy while reaffirming that the primary role of RM lies in explainable attribution rather than standalone ranking accuracy.

7.3.3. Convergent Validity Across Simulation Oracles

To test construct validity, we compared node criticality rankings across our three simulation engines: the topological cascade reachability oracle $I^*(v)$ (`FaultInjector`), the multi-metric composite oracle $I_{\text{comp}}(v)$ (`FailureSimulator`), and the discrete-event queue-flow simulator $I_{\text{dyn}}(v)$ (`MessageFlowSimulator`).

Across all seven synthetic scenarios, the behavioural SimPy simulator (I_{dyn}) and the topological cascade reachability oracle (I^*) exhibited strong rank convergence:

- I_{dyn} vs. $I^*(v)$: Mean Spearman $\rho = \mathbf{0.7654}$ (ranging from 0.5485 on Hub-and-Spoke to 0.9378 on Healthcare), with mean Kendall $\tau = 0.630$ and top- K Jaccard overlap of 0.316.
- I_{comp} vs. I_{dyn} : Mean Spearman $\rho = 0.4650$ (ranging from 0.1208 to 0.6574).
- I_{comp} vs. $I^*(v)$: Mean Spearman $\rho = 0.3943$ (ranging from 0.0920 to 0.5777).

This strong agreement between independent behavioural queue simulations and structural cascade reachability confirms that our continuous target label $I^*(v)$ captures genuine dynamic service disruption.

7.3.4. Domain-Specific Weighting Sensitivity

We investigated the impact of Context of Use reweighting $\vec{\omega} = [q_R, q_M]^\top$ across 10 evaluation scenarios. Domain-specific reweighting $Q_{\text{domain}}(v)$ improved rank correlation by an average of $\Delta\rho = +\mathbf{0.0265}$ over static uniform weighting, maintaining an average Kendall rank fidelity of $\tau = 0.9677$. This demonstrates that operational Context of Use can be tuned without distorting underlying structural stability.

7.3.5. Threshold and Normalization Sensitivity

Sweeping 7 scenarios $\times$ 7 tier thresholds ($z \in [1.0, 2.5]$, IQR multipliers) $\times$ 3 normalization techniques (min-max, standard, rank-quantile) revealed that tier categorization is primarily sensitive to threshold selection ($\Delta\rho = 0.1896$ across extreme thresholds), whereas ranking performance is exceptionally stable across normalization regimes ($\Delta\rho = 0.0158$).

7.3.6. Anti-Pattern Detection and CI/CD Quality Gates

Validating our rule-based architectural anti-pattern catalog against the multi-metric composite oracle $I_{\text{comp}}(v)$ yielded a mean detection $F_1 = 0.3781$ across 8 system scenarios. Analysis execution times ranged from 0.04 s (small scenarios, 50 nodes) to 54.85 s (enterprise event meshes, 300 nodes), confirming that SaG runs comfortably within standard pre-commit and pull-request CI/CD quality gate budgets.

7.3.7. HGT Attention Weight Analysis

Extracting relational attention matrices from the trained HGT model on the ATM Case Study (Figure 3) revealed that the model places the highest attention weights ($\alpha_{uv} > 0.45$) on downstream PUBLISHES_TO and CALLS dependencies connecting shared persistence brokers and authorization libraries, directly reflecting empirical cascade pathways.

7.4. RQ4: Real-World Distributed Software Architecture Validation

To assess external validity on authentic production architectures, we evaluated SaG on three open-source distributed systems.

Table 9: Empirical validation on authentic real-world distributed software architectures.

Real-World Architecture	$ V $	$ V_{\text{app}} $	Spearman ρ	Kendall τ	$F_1@K$	Tie-Robust F_1	Non-Zero	Gain
Autware.universe (ROS 2)	75	32	0.688 ± 0.009	0.517	0.800	0.800	19 / 32	+0.36
Cloud Microservices Mesh	60	22	0.778 ± 0.001	0.639	1.000	0.760	8 / 22	+0.01
Train-Ticket Booking Mesh	90	41	0.759 ± 0.001	0.605	1.000	0.810	14 / 41	+0.26

Key Insights for RQ4:

1. **Strong Real-World Rank Agreement:** SaG achieves high rank correlation on Cloud Microservices ($\rho = 0.778$) and Train-Ticket ($\rho = 0.759$), and solid agreement on Autware.universe ($\rho = 0.688$).
2. **Critical-Set Containment:** Every single application with non-zero cascading impact in Cloud Microservices and Train-Ticket is successfully captured within the predicted top- K critical set (tie-robust $F_1@K = 0.760$ and 0.810).
3. **Substantial Predictive Gain:** SaG outperforms raw degree centrality by +0.360 on Autware and +0.264 on Train-Ticket, demonstrating that typed dependency derivation captures critical architectural semantics beyond superficial connectivity.

8. Discussion, Threats to Validity, and Conclusion

8.1. Discussion and Practical Implications

Our empirical findings provide clear guidance for software architects and reliability engineers:

- **When to Use Graph Learning:** Heterogeneous GNNs (HGL) provide the greatest return when the objective is identifying the critical top- K shortlist of components for architectural hardening ($F_1@K = 0.465$), and when evaluating new, unseen system architectures out of distribution ($\rho = 0.608$).
- **When to Use Interpretable Attribution:** For root-cause diagnosis and remediation planning, the deterministic RM profile is indispensable. Distinguishing whether a service is critical due to single-point-of-failure exposure (Availability) or wide error propagation (Fault Tolerance) directly dictates whether to introduce load-balanced replicas or circuit-breaker policies.

8.2. Threats to Validity

- **Construct Validity:** Our ground truth is generated via discrete-event cascade simulation on structural models rather than observing live production outages. To mitigate construct divergence, we evaluated multiple independent simulation engines (**FaultInjector**, **FailureSimulator**, **MessageFlowSimulator**) and confirmed strong convergent validity between dynamic queue-flow simulation and cascade reachability ($\rho = 0.765$).
- **Internal Validity:** Potential feature leakage is prevented by strict graph view separation: predictors operate on G_{analysis} , whereas ground-truth simulators operate on $G_{\text{structural}}$.
- **External Validity:** While our corpus spans ten distinct system domains and three authentic open-source architectures (ROS 2 and microservices), future work should evaluate larger enterprise deployments with hundreds of microservices.
- **Conclusion Validity:** Given the heavy-tailed, non-normal distribution of cascading failure impacts, all statistical comparisons utilize non-parametric rank correlation (Spearman ρ , Kendall τ), bootstrap confidence intervals ($B = 2,000$), and paired Wilcoxon signed-rank tests.

8.3. Limitations and Future Work

1. **Safety and Security Integration:** SaG currently focuses on Reliability and Maintainability. Incorporating formal hazard categories (e.g., ISO 26262 ASIL ratings) and security threat models into the multigraph schema represents an important direction for future research.
2. **Hardware-in-the-Loop (HIL) Validation:** Validating SaG’s predictions against real-time physical fault-injection testbeds in cyber-physical environments.
3. **Automated Architectural Refactoring:** Extending SaG from predictive analysis to prescriptive synthesis—automatically generating pull requests that reconfigure QoS policies and add redundancy.

8.4. Conclusion

We presented **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that models complex distributed pub-sub and microservice architectures as typed, weighted multigraphs. By combining relation-specific Heterogeneous Graph Transformers with an interpretable Reliability–Maintainability quality attribution model, SaG bridges the Architecture–Code Gap, forecasting cascading failure risks before systems are deployed.

Our empirical results across synthetic and authentic open-source distributed systems demonstrate that heterogeneous graph learning provides significant advantages in identifying critical components ($F_1@K = 0.465$) and generalizing across unseen architectures ($\rho = 0.608$), while delivering actionable architectural diagnostics for resilient distributed systems engineering.

CRediT authorship contribution statement

[Omitted for double-anonymised review. To be completed on acceptance with the standard CRediT roles: Conceptualization; Methodology; Software; Validation; Formal analysis; Investigation; Data curation; Writing – original draft; Writing – review and editing; Visualization; Supervision.]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[Omitted for double-anonymised review.]

Data availability

The replication package—including the seven synthetic scenario datasets, topology generators, simulation harnesses, and real-world architecture adapters—will be made openly available upon publication.

Declaration of generative AI use

[To be completed by the authors in accordance with the journal’s policy.]

References

- [1] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [2] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [3] OASIS, MQTT version 5.0, OASIS Standard (2019).
- [4] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [5] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [6] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [7] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [8] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering* SE-2 (4) (1976) 308–320.
- [9] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.

- [10] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [11] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [12] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [13] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [14] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [15] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [16] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [17] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems* 30 (NeurIPS), 2017, pp. 1024–1034.
- [18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [19] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
- [20] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 580–586.
- [21] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: *Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE)*, 2007.
- [22] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
- [23] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [24] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
- [25] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: *Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA)*, LNCS 5581, 2009, pp. 146–162.
- [26] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [27] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
- [28] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, *Microservices: Yesterday, today, and tomorrow*, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [29] W. Cunningham, The WyCash portfolio management system, in: *Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 1992, pp. 29–30.

- [30] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [31] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [32] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [33] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [34] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [35] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [36] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [37] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [38] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.
- [39] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710.
- [40] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2331–2341.
- [41] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [42] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
- [43] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.
- [44] Team SimPy, Simpy: Event discrete simulation for Python, <https://simpy.readthedocs.io> (2020).
- [45] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: *Proc. 25th Int. Conf. on Machine Learning (ICML)*, 2008, pp. 1192–1199.
- [46] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: *Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS)*, 2018, pp. 287–296.
- [47] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, *Software artifact* (2024).

- 628 [48] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice
629 systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software*
630 *Engineering* 47 (2) (2021) 243–260.
- 631 [49] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- 632 [50] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- 633 [51] C. Spearman, The proof and measurement of association between two things, *American Journal of*
634 *Psychology* 15 (1) (1904) 72–101.